

DATA COLLECTION

1. Importing required Libraries for EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline
```

1. Loading CSV data as Pandas DataFrame

```
df=pd.read_csv('data_set.csv')
```

1. Types of variables

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18434 entries, 0 to 18433
Columns: 141 entries, uniqueid to adblue_level
dtypes: float64(122), int64(13), object(6)
memory usage: 19.8+ MB

num=[feature for feature in df.columns if df[feature].dtype=='int64'
or df[feature].dtype=='float64']
print('We have {} numerical features: {}'.format(len(num), num))
cat=[feature for feature in df.columns if df[feature].dtype=='object']
print('\nWe have {} categorical features: {}'.format(len(cat), cat))

We have 135 numerical features: ['ts', 'lat', 'lng',
'external_bat_voltage', 'internal_bat_voltage', 'engineload',
'tanklevel', 'throttle', 'coolant', 'intakeairtemp', 'engineoiltemp',
'imap', 'vehiclespeed', 'warmups', 'rpm', 'obddistance', 'maf',
'fuelrail', 'runtime', 'mil', 'obdstandards', 'telltale',
'pto_drive_engagement', 'engine_torque_percent', 'service_distance',
'gross_combination_vehicle_weight', 'selected_gear', 'current_gear',
'fuel_consumption', 'fuel_level', 'fl_level', 'fuel_rate',
'fuel_economy', 'accelerator_pedal_pos', 'vehicle_weight',
'air_suspension_pressure_front_axle_left',
'air_suspension_pressure_front_axle_right',
'air_suspension_pressure_rear_axle_left',
'air_suspension_pressure_rear_axle_right', 'state_of_charge',
'battery_pack_voltage', 'battery_pack_current', 'power',
'lamp_status', 'hydraulic_oil_filter', 'air_filter', 'no_charging',
'low_fuel', 'engine_temp_high', 'hydraulic_oil_temp_high',
'water_in_fuel', 'low_oil_pressure', 'roller_speed',
'vibration_status', 'axle_position', 'tire_position', 'tire_pressure',
```

```
'tire_temperature', 'tire_air_leakage_rate',
'tire_sensor_enable_status', 'tire_status',
'tire_sensor_electric_fault', 'extended_tire_pressure_support',
'tire_pressure_threshold_detection', 'extended_tire_pressure',
'cpc_system_type', 'required_tire_pressure', 'cpc_tire_id',
'cpc_graphical_position', 'cpc_tire_location', 'cpc_ttm_id',
'barometric_pressure', 'secondary_air_status',
'catalyst_temperature_bank1_sensor1',
'catalyst_temperature_bank1_sensor2', 'distance_since_mil',
'commanded_egr', 'egr_error', 'drive_cycle_status',
'ambient_air_temperature', 'fuel_system_status',
'short_term_fuel_trim_bank_1', 'long_term_fuel_trim_bank_1',
'timing_advance', 'oxygen_sensors_present', 'oxygen_sensor_1_voltage',
'oxygen_sensor_2_voltage', 'commanded_evaporative_purge',
'control_module_voltage', 'absolute_load_value',
'relative_throttle_position', 'oxygen_sensor_1_short_term_fuel_trim',
'oxygen_sensor_2_short_term_fuel_trim', 'engine_throttle_valve1_pos',
'engine_throttle_valve2_pos', 'enginefueltemp',
'engine_airlet_pressure', 'engine_oil_pressure',
'drivers_demand_engine_torque_percent', 'engine_torque_mode',
'accelerator_pedal_pos_2', 'engine_pto_switch_status', 'clientid',
'alerttype', 'alertvalue', 'from_ts', 'to_ts', 'server_ts', 'to_lat',
'to_lng', 'tripid', 'alert_completed', 'view_status',
'scheduled_start_loc', 'scheduled_end_loc', 'vehiclenunder',
'areaandroute', 'type', 'breached_in_userid', 'alerted_to_in_userid',
'breached_out_userid', 'alerted_to_out_userid',
'alert_generated_by_app_userid', 'alert_ended_by_app_userid',
'request_id', 'live_location', 'to_be_expired_at', 'status',
'event_id', 'particulated_trap_inlet_pressure', 'boost_pressure',
'intake_manifold_1_temperature', 'air_inlet_pressure',
'exhaust_gas_temperature', 'adblue_level']
```

We have 6 categorical features: ['uniqueid', 'pluscode',
'can_raw_data', 'brake_switch_status', 'clutch_switch_status',
'parking_switch_status']

1. Number of Rows and Columns respectively

```
df.shape
```

```
(18434, 141)
```

1. Show top 5 records

```
df.head()
```

	uniqueid	ts	lat	lng
external_bat_voltage \				
0 it_220403501	1709340731	19.069340	78.355148	
NaN				

```

1  it_220403501  1709340794  19.069347  78.355141
NaN
2  it_220403501  1709340857  19.069414  78.355179
NaN
3  it_220403501  1709340920  19.069258  78.355225
NaN
4  it_220403501  1709340983  19.069094  78.355339
NaN

```

```

      internal_bat_voltage  engineload  tanklevel  throttle  coolant  ...
\
0      NaN                28          NaN      NaN      33      ...
1      NaN                22          NaN      NaN      41      ...
2      NaN                25          NaN      NaN      48      ...
3      NaN                35          NaN      NaN      52      ...
4      NaN                0           NaN      NaN      57      ...

```

```

      live_location  to_be_expired_at  status  event_id  \
0      NaN          NaN          NaN      NaN      NaN
1      NaN          NaN          NaN      NaN      NaN
2      NaN          NaN          NaN      NaN      NaN
3      NaN          NaN          NaN      NaN      NaN
4      NaN          NaN          NaN      NaN      NaN

```

```

      particulated_trap_inlet_pressure  boost_pressure  \
0      NaN          NaN
1      NaN          NaN
2      NaN          NaN
3      NaN          NaN
4      NaN          NaN

```

```

      intake_manifold_1_temperature  air_inlet_pressure
exhaust_gas_temperature  \
0      NaN          NaN
NaN
1      NaN          NaN
NaN
2      NaN          NaN
NaN
3      NaN          NaN
NaN
4      NaN          NaN
NaN

```

```

      adblue_level

```

0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 141 columns]

DATA CLEANING

1. Removing unnecessary features from dataset

```
# Drop columns with all NaN values
df_cleaned = df.dropna(axis=1, how='all')
# Drop columns with the same value
df_cleaned = df_cleaned.loc[:, df_cleaned.nunique() > 1]

num=[feature for feature in df_cleaned.columns if
df_cleaned[feature].dtype=='int64' or
df_cleaned[feature].dtype=='float64']
print('We have {} numerical features: {}'.format(len(num), num))
cat=[feature for feature in df_cleaned.columns if
df_cleaned[feature].dtype=='object']
print('\nWe have {} categorical features: {}'.format(len(cat), cat))
```

We have 21 numerical features: ['ts', 'lat', 'lng', 'engineload', 'coolant', 'vehiclespeed', 'rpm', 'obddistance', 'runtime', 'engine_torque_percent', 'current_gear', 'fuel_consumption', 'fuel_level', 'fl_level', 'fuel_rate', 'fuel_economy', 'accelerator_pedal_pos', 'engine_throttle_valve_pos', 'drivers_demand_engine_torque_percent', 'engine_torque_mode', 'adblue_level']

We have 5 categorical features: ['pluscode', 'can_raw_data', 'brake_switch_status', 'clutch_switch_status', 'parking_switch_status']

df_cleaned.head()

	ts	lat	lng	engineload	coolant	vehiclespeed
0	1709340731	19.069340	78.355148	28	33	0.00000
1	1709340794	19.069347	78.355141	22	41	0.00000
2	1709340857	19.069414	78.355179	25	48	1.40625
3	1709340920	19.069258	78.355225	35	52	2.81250
4	1709340983	19.069094	78.355339	0	57	4.53125

	rpm	obddistance	runtime	engine_torque_percent	...	\
0	749.000	60780800	1970.85	21	...	
1	1208.375	60780800	1970.85	18	...	
2	752.625	60780800	1970.85	18	...	
3	996.375	60780820	1970.90	27	...	
4	1065.750	60780840	1970.90	1	...	

	accelerator_pedal_pos	pluscode	\
0	0.0	7JFW3994+	
1	23.6	7JFW3994+	
2	0.0	7JFW3994+	
3	33.6	7JFW3994+	
4	3.2	7JFW3994+	

	can_raw_data	\
0	0CF00300:D1001CFFFF4C4886,0CF00400:407D9268170...	
1	0CF00300:D03B16FFFF4C4283,0CF00400:41908FC3250...	
2	0CF00300:D10019FFFF4C4983,0CF00400:307D8F85170...	
3	0CF00300:D05423FFFF4C4C82,0CF00400:519998231F0...	
4	0CF00300:D10800FFFF4C3284,0CF00400:017E7E4E210...	

	engine_throttle_valve1_pos	drivers_demand_engine_torque_percent	\
0	100.0	0	
1	99.6	19	
2	99.6	0	
3	99.6	28	
4	100.0	1	

	engine_torque_mode	brake_switch_status	clutch_switch_status	\
0	1	Released	Released	
1	2	Released	Pressed	
2	1	Pressed	Pressed	
3	2	Released	Pressed	
4	2	Released	Pressed	

	parking_switch_status	adblue_level
0	Pressed	NaN
1	Pressed	NaN
2	Released	NaN
3	Released	NaN
4	Released	NaN

[5 rows x 26 columns]

```
df_cleaned.drop_duplicates()
df_cleaned.shape
```

(18434, 26)

```
df_cleaned.isnull().sum()
```

ts	0
lat	0
lng	0
engineload	0
coolant	0
vehiclespeed	0
rpm	0
obddistance	0
runtime	14
engine_torque_percent	0
current_gear	0
fuel_consumption	0
fuel_level	2
fl_level	0
fuel_rate	0
fuel_economy	0
accelerator_pedal_pos	0
pluscode	0
can_raw_data	0
engine_throttle_valve1_pos	0
drivers_demand_engine_torque_percent	0
engine_torque_mode	0
brake_switch_status	0
clutch_switch_status	0
parking_switch_status	0
adblue_level	3664
dtype:	int64

```
df_cleaned['runtime'].mode()[0]
```

2260.25

```
df_cleaned['runtime']=df_cleaned['runtime'].fillna(df_cleaned['runtime'].mode()[0])
```

```
df_cleaned['fuel_level']=df_cleaned['fuel_level'].fillna(df_cleaned['fuel_level'].mode()[0])
```

```
df_cleaned['adblue_level']=df_cleaned['adblue_level'].fillna(df_cleaned['adblue_level'].mode()[0])
```

```
df_cleaned.isnull().sum()
```

ts	0
lat	0
lng	0
engineload	0
coolant	0
vehiclespeed	0
rpm	0
obddistance	0
runtime	0

```

engine_torque_percent      0
current_gear               0
fuel_consumption           0
fuel_level                 0
fl_level                   0
fuel_rate                  0
fuel_economy               0
accelerator_pedal_pos      0
pluscode                   0
can_raw_data               0
engine_throttle_valve1_pos 0
drivers_demand_engine_torque_percent 0
engine_torque_mode         0
brake_switch_status        0
clutch_switch_status       0
parking_switch_status      0
adblue_level               0
dtype: int64

```

```
df_cleaned.head()
```

	ts	lat	lng	engineload	coolant	vehiclespeed
0	1709340731	19.069340	78.355148	28	33	0.00000
1	1709340794	19.069347	78.355141	22	41	0.00000
2	1709340857	19.069414	78.355179	25	48	1.40625
3	1709340920	19.069258	78.355225	35	52	2.81250
4	1709340983	19.069094	78.355339	0	57	4.53125

	rpm	obddistance	runtime	engine_torque_percent	...	\
0	749.000	60780800	1970.85	21	...	
1	1208.375	60780800	1970.85	18	...	
2	752.625	60780800	1970.85	18	...	
3	996.375	60780820	1970.90	27	...	
4	1065.750	60780840	1970.90	1	...	

	accelerator_pedal_pos	pluscode	\
0	0.0	7JFW3994+	
1	23.6	7JFW3994+	
2	0.0	7JFW3994+	
3	33.6	7JFW3994+	
4	3.2	7JFW3994+	

	can_raw_data	\
0	0CF00300:D1001CFFFF4C4886,0CF00400:407D9268170...	

```

1 0CF00300:D03B16FFFF4C4283,0CF00400:41908FC3250...
2 0CF00300:D10019FFFF4C4983,0CF00400:307D8F85170...
3 0CF00300:D05423FFFF4C4C82,0CF00400:519998231F0...
4 0CF00300:D10800FFFF4C3284,0CF00400:017E7E4E210...

```

	engine_throttle_valve1_pos	drivers_demand_engine_torque_percent	\
0	100.0	0	
1	99.6	19	
2	99.6	0	
3	99.6	28	
4	100.0	1	

	engine_torque_mode	brake_switch_status	clutch_switch_status	\
0	1	Released	Released	
1	2	Released	Pressed	
2	1	Pressed	Pressed	
3	2	Released	Pressed	
4	2	Released	Pressed	

	parking_switch_status	adblue_level
0	Pressed	100.0
1	Pressed	100.0
2	Released	100.0
3	Released	100.0
4	Released	100.0

[5 rows x 26 columns]

1. Converting all binary data into integer value

```

print(len(df_cleaned['brake_switch_status'].unique()))
print(len(df_cleaned['clutch_switch_status'].unique()))
print(len(df_cleaned['parking_switch_status'].unique()))

print(len(df_cleaned['pluscode'].unique()))
print(len(df_cleaned['can_raw_data'].unique()))

2
2
2
2639
17874

df_2=df_cleaned.copy()
df_cleaned['brake_switch_status']=df_cleaned['brake_switch_status'].map(
{'Pressed':0,'Released':1})
df_cleaned['clutch_switch_status']=df_cleaned['clutch_switch_status'].
map({'Pressed':0,'Released':1})
df_cleaned['parking_switch_status']=df_cleaned['parking_switch_status']
.map({'Pressed':0,'Released':1})

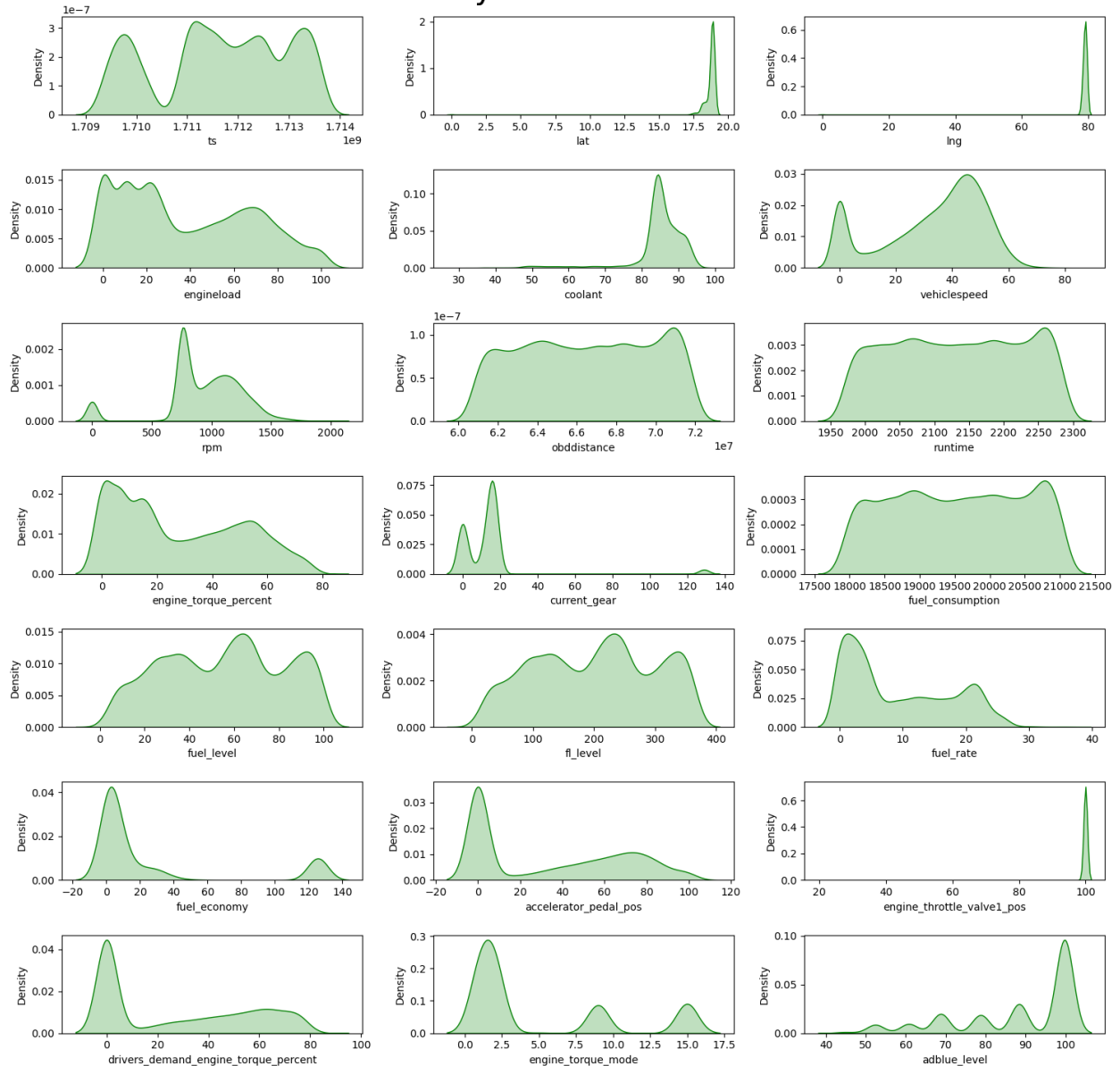
```

UNIVARIATE ANALYSIS

1. Numerical Data I. Plotting Kernel Density Estimate plot

```
plt.figure(figsize=(15,15))
plt.suptitle('Univariate Analysis of Numerical Features', fontsize=30,
fontweight=10)
for i in range(0, len(num)):
    plt.subplot(7,3,i+1)
    sns.kdeplot(x=df_2[num[i]],shade=True,color='g')
    plt.xlabel(num[i])
    plt.tight_layout()
```

Univariate Analysis of Numerical Features



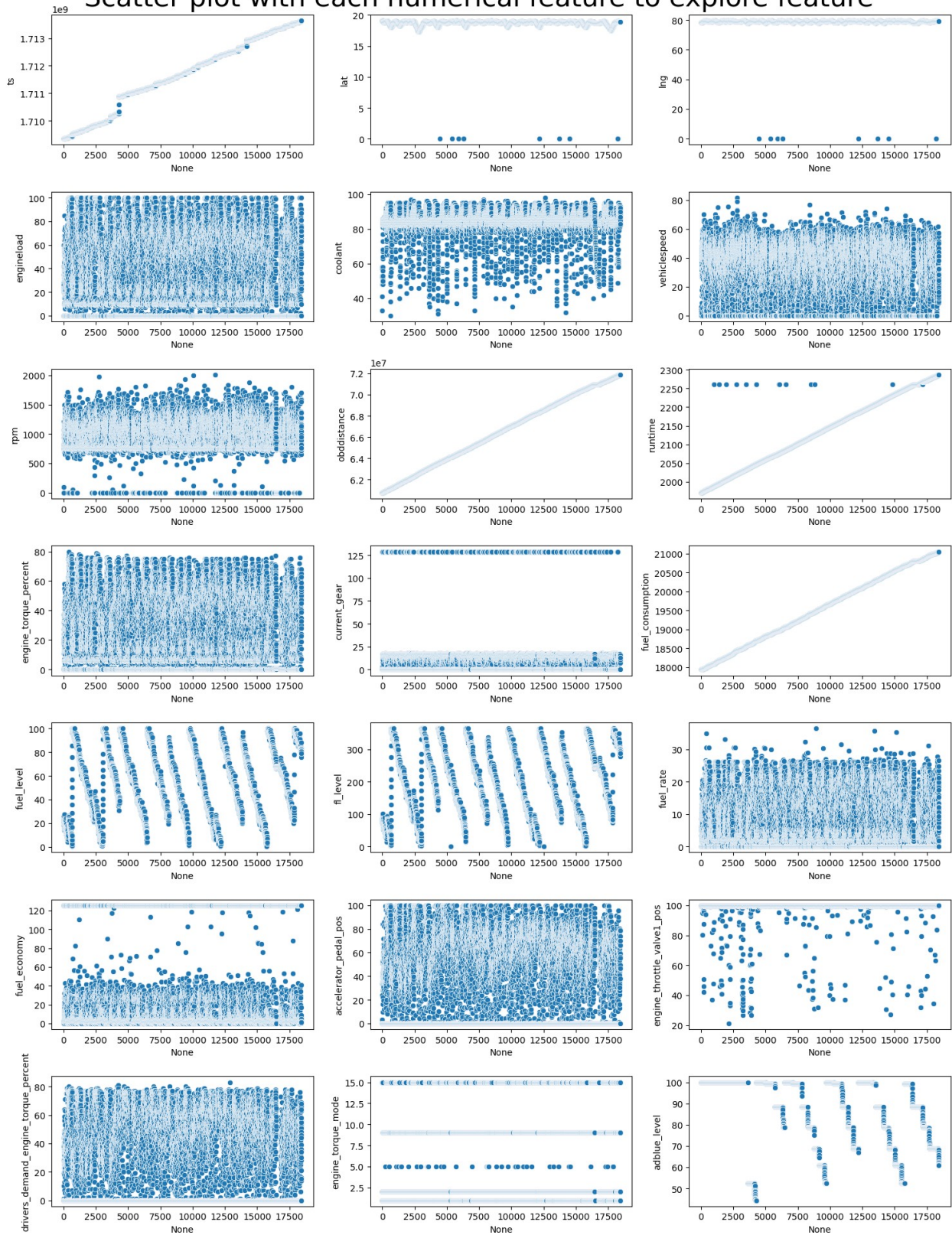
Observations-1:

1. runtime, fuel_consumption and obddistance has similar graphs and can be combined to form a new single feature mileage which is similar to fuel_economy.
2. drivers_demand_engine_torque_percent and accelerator_pedal_pos has similar graphs and can be combined to form a new feature driver_input_demand
3. fuel_level and fl_level are same thing and one can be omitted
4. engine_torque_percent and engineload has similar graphs and can be combined to form a new feature performance_demand

II. Plotting Scatter plot

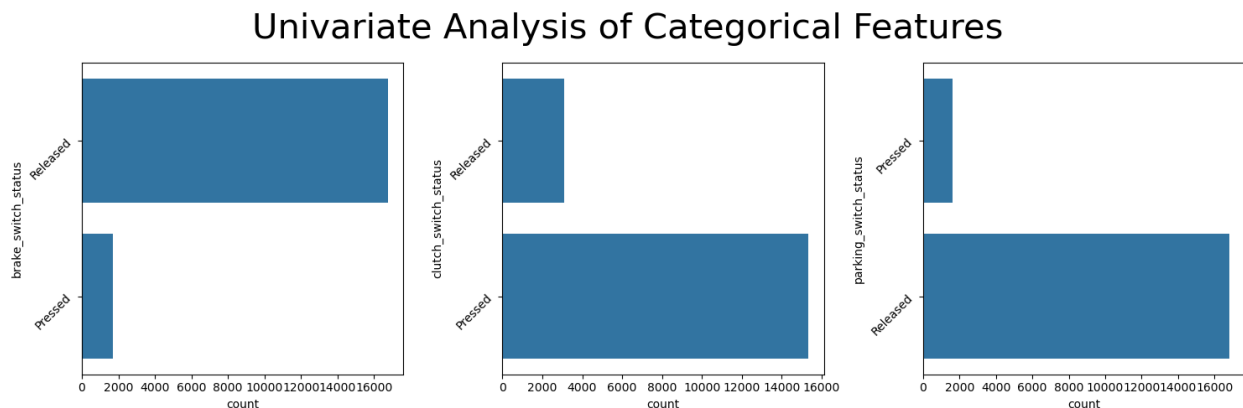
```
plt.figure(figsize=(15,20))
plt.suptitle('Scatter plot with each numerical feature to explore
feature', fontsize=30, fontweight=10)
for i in range(0, len(num)):
    plt.subplot(7,3,i+1)
    sns.scatterplot(y=num[i],x=df_2.index, data=df_2)
plt.tight_layout()
```

Scatter plot with each numerical feature to explore feature



1. Categorical Data I. Plotting Count plot

```
plt.figure(figsize=(15,5))
plt.suptitle('Univariate Analysis of Categorical Features',
fontsize=30, fontweight=10)
cat1=['brake_switch_status', 'clutch_switch_status',
'parking_switch_status']
for i in range(0, len(cat1)):
    plt.subplot(1,3,i+1)
    sns.countplot(y=df_2[cat1[i]])
    plt.ylabel(cat1[i])
    plt.yticks(rotation=45)
    plt.tight_layout()
```



Observations-2:

1. brake_switch_status is mostly released
2. clutch_switch_status is mostly pressed
3. parking_switch_status is mostly released

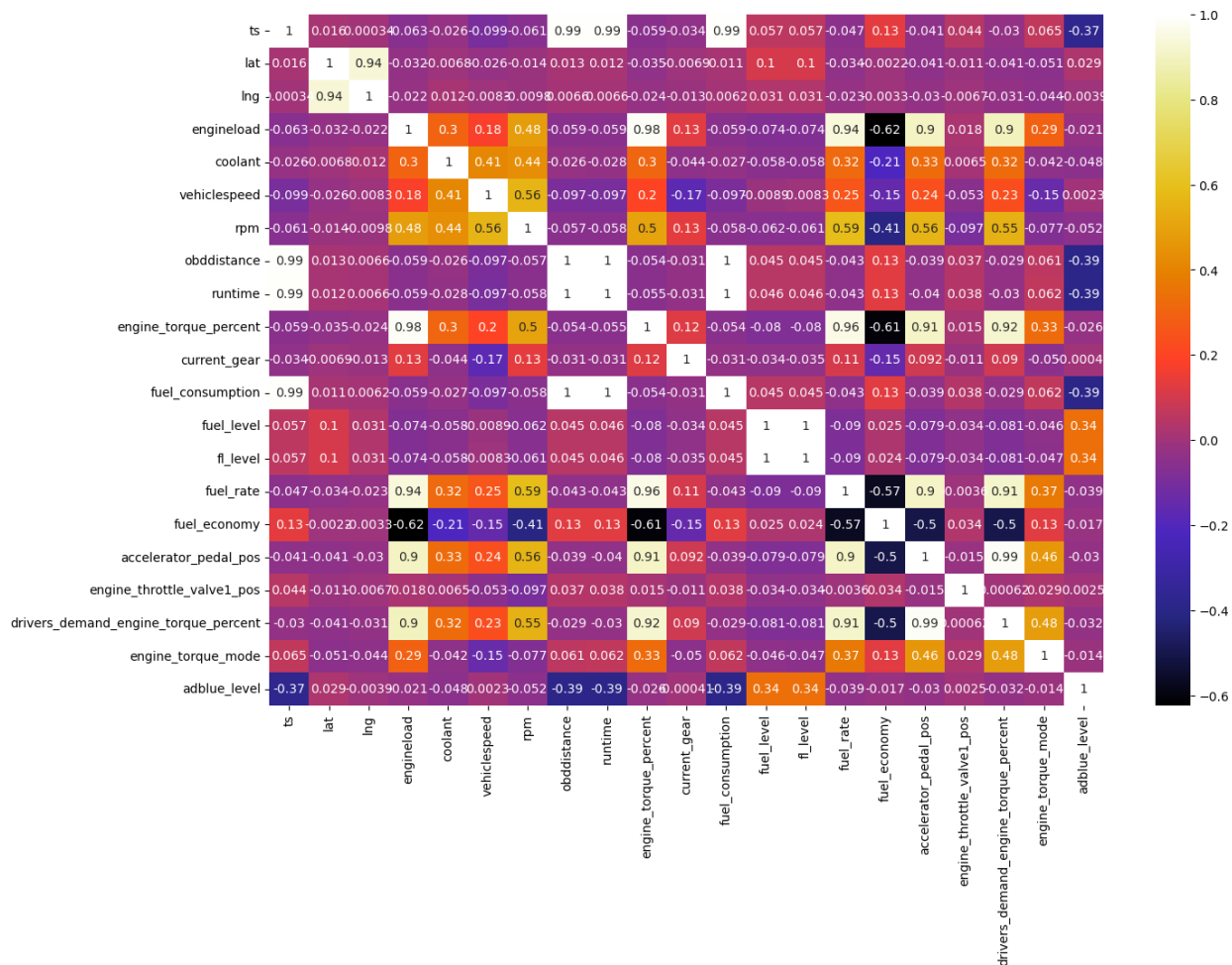
BIVARIATE ANALYSIS

1. Plotting Heatmap

```
col=['can_raw_data', 'pluscode', 'parking_switch_status', 'clutch_switch_
status', 'brake_switch_status'] #Deleting all categorical data
for col1 in col:
    del df_2[col1]

df_corr=df_2.corr() #correlation matrix

plt.figure(figsize=(15,10))
sns.heatmap(df_corr, cmap="CMRmap", annot=True) #plotting heatmap
plt.show()
```



```
mask = np.tril(np.ones_like(np.abs(df_corr), dtype=bool), k=-1)
#masking lower triangle of the matrix
df_pairs = df_corr.where(mask & (np.abs(df_corr) > 0.4)).stack()
#forming pairs having absolute value of Correlation coefficient
greater than 0.4
pairs_with_corr = df_pairs.reset_index()
pairs_with_corr.columns = ['Feature1', 'Feature2', 'Correlation']
pairs_with_corr
```

```

Feature1 \
0 lng
1 vehiclespeed
2 rpm
3 rpm
4 rpm
5 obddistance
6 runtime
7 runtime
8 engine_torque_percent
9 engine_torque_percent
```

10		fuel_consumption
11		fuel_consumption
12		fuel_consumption
13		fl_level
14		fuel_rate
15		fuel_rate
16		fuel_rate
17		fuel_economy
18		fuel_economy
19		fuel_economy
20		fuel_economy
21		accelerator_pedal_pos
22		accelerator_pedal_pos
23		accelerator_pedal_pos
24		accelerator_pedal_pos
25		accelerator_pedal_pos
26	drivers_demand_engine_torque_percent	
27	drivers_demand_engine_torque_percent	
28	drivers_demand_engine_torque_percent	
29	drivers_demand_engine_torque_percent	
30	drivers_demand_engine_torque_percent	
31	drivers_demand_engine_torque_percent	
32		engine_torque_mode
33		engine_torque_mode

	Feature2	Correlation
0	lat	0.935495
1	coolant	0.412702
2	engineload	0.482889
3	coolant	0.435605
4	vehiclespeed	0.559204
5	ts	0.993215
6	ts	0.991354
7	obddistance	0.998241
8	engineload	0.982212
9	rpm	0.504514
10	ts	0.992975
11	obddistance	0.999959
12	runtime	0.998248
13	fuel_level	0.999650
14	engineload	0.939175
15	rpm	0.586125
16	engine_torque_percent	0.959265
17	engineload	-0.623559
18	rpm	-0.411809
19	engine_torque_percent	-0.609425
20	fuel_rate	-0.572712
21	engineload	0.902727
22	rpm	0.560249

23	engine_torque_percent	0.910516
24	fuel_rate	0.904892
25	fuel_economy	-0.504534
26	engineload	0.897413
27	rpm	0.545234
28	engine_torque_percent	0.923056
29	fuel_rate	0.912270
30	fuel_economy	-0.498786
31	accelerator_pedal_pos	0.985564
32	accelerator_pedal_pos	0.463396
33	drivers_demand_engine_torque_percent	0.484400

1. Plotting boxplot

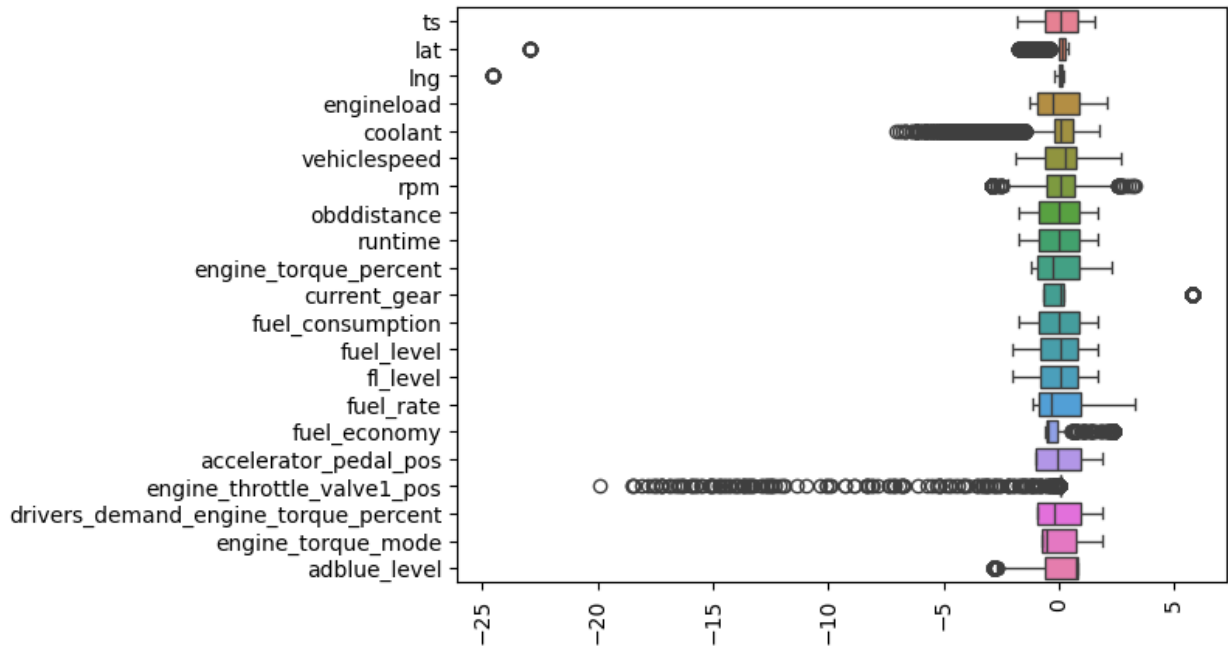
```

from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
df_scaled=scaler.fit_transform(df_2)
normalized_df = pd.DataFrame(df_scaled, columns=df_2.columns) #scaling
data

sns.boxplot(data= normalized_df, orient="h")
plt.xticks(rotation=90)

(array([-30., -25., -20., -15., -10., -5., 0., 5., 10.]),
 [Text(-30.0, 0, '-30'),
  Text(-25.0, 0, '-25'),
  Text(-20.0, 0, '-20'),
  Text(-15.0, 0, '-15'),
  Text(-10.0, 0, '-10'),
  Text(-5.0, 0, '-5'),
  Text(0.0, 0, '0'),
  Text(5.0, 0, '5'),
  Text(10.0, 0, '10')])

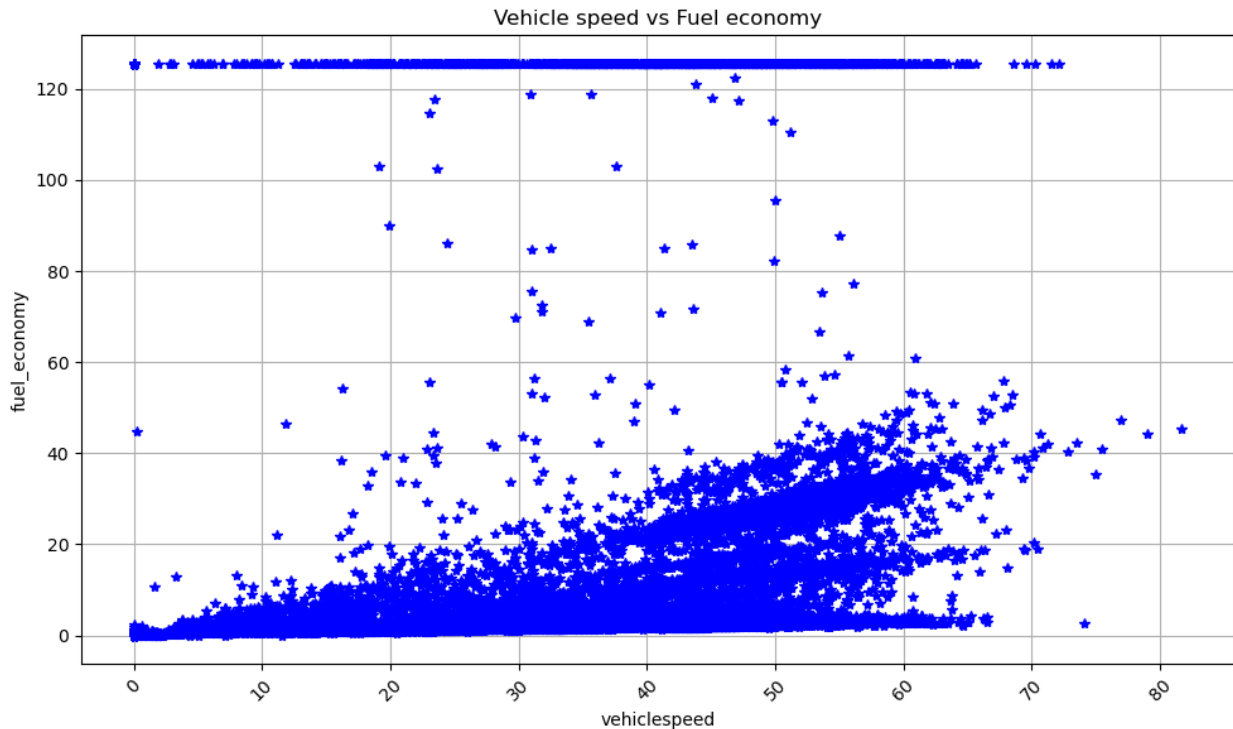
```



Vehicle Performance

1. Plotting graph between Vehicle speed and Fuel economy

```
plt.figure(figsize=(10, 6))
plt.plot(df_2['vehiclespeed'], df_2['fuel_economy'], marker='*',
linestyle='-', color='b')
plt.title('Vehicle speed vs Fuel economy')
plt.xlabel('vehiclespeed')
plt.ylabel('fuel_economy')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Observation-3:

1. The graph shows there are 2 groups of data, so we need to separate them into 2 groups and then analyse

```
from sklearn.cluster import KMeans
scaler = StandardScaler()
scaled_data =
scaler.fit_transform(df_2[['fuel_economy','vehiclespeed']])
k_means = KMeans(n_clusters=2, random_state=0)
pred=k_means.fit(scaled_data)
df_2['Cluster'] = pred.labels_ #clustered into 2 groups and added the
column to the dataframe
df_2.head() #view first 5 rows
```

	ts	lat	lng	engineload	coolant	vehiclespeed
0	1709340731	19.069340	78.355148	28	33	0.00000
1	1709340794	19.069347	78.355141	22	41	0.00000
2	1709340857	19.069414	78.355179	25	48	1.40625
3	1709340920	19.069258	78.355225	35	52	2.81250
4	1709340983	19.069094	78.355339	0	57	4.53125

rpm obddistance runtime engine_torque_percent ...

	fuel_level	\
0	749.000	60780800 1970.85 21 ...
23.6		
1	1208.375	60780800 1970.85 18 ...
23.6		
2	752.625	60780800 1970.85 18 ...
22.8		
3	996.375	60780820 1970.90 27 ...
20.4		
4	1065.750	60780840 1970.90 1 ...
23.2		

	fl_level	fuel_rate	fuel_economy	accelerator_pedal_pos	\
0	86.14	5.30	0.000000	0.0	
1	86.14	6.45	0.000000	23.6	
2	83.22	4.70	0.337891	0.0	
3	74.46	8.60	0.121094	33.6	
4	84.68	0.00	125.500000	3.2	

	engine_throttle_valve_pos	drivers_demand_engine_torque_percent	\
0	100.0	0	
1	99.6	19	
2	99.6	0	
3	99.6	28	
4	100.0	1	

	engine_torque_mode	adblue_level	Cluster
0	1	100.0	1
1	2	100.0	1
2	1	100.0	1
3	2	100.0	1
4	2	100.0	0

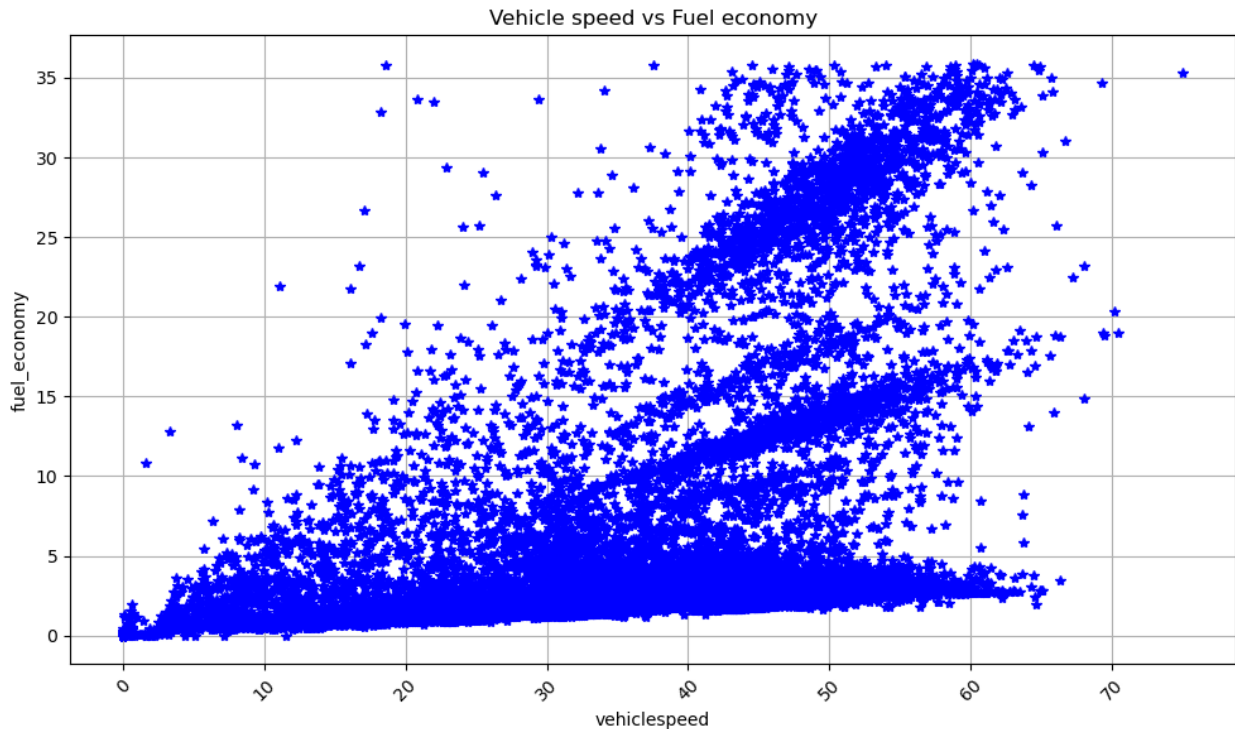
[5 rows x 22 columns]

```
df_5=df_2.where(df_2['Cluster']==1).dropna()
del df_5['Cluster'] #extracting the data from the dataframe with
cluster value 1(as cluster with value 1 shows a pattern)

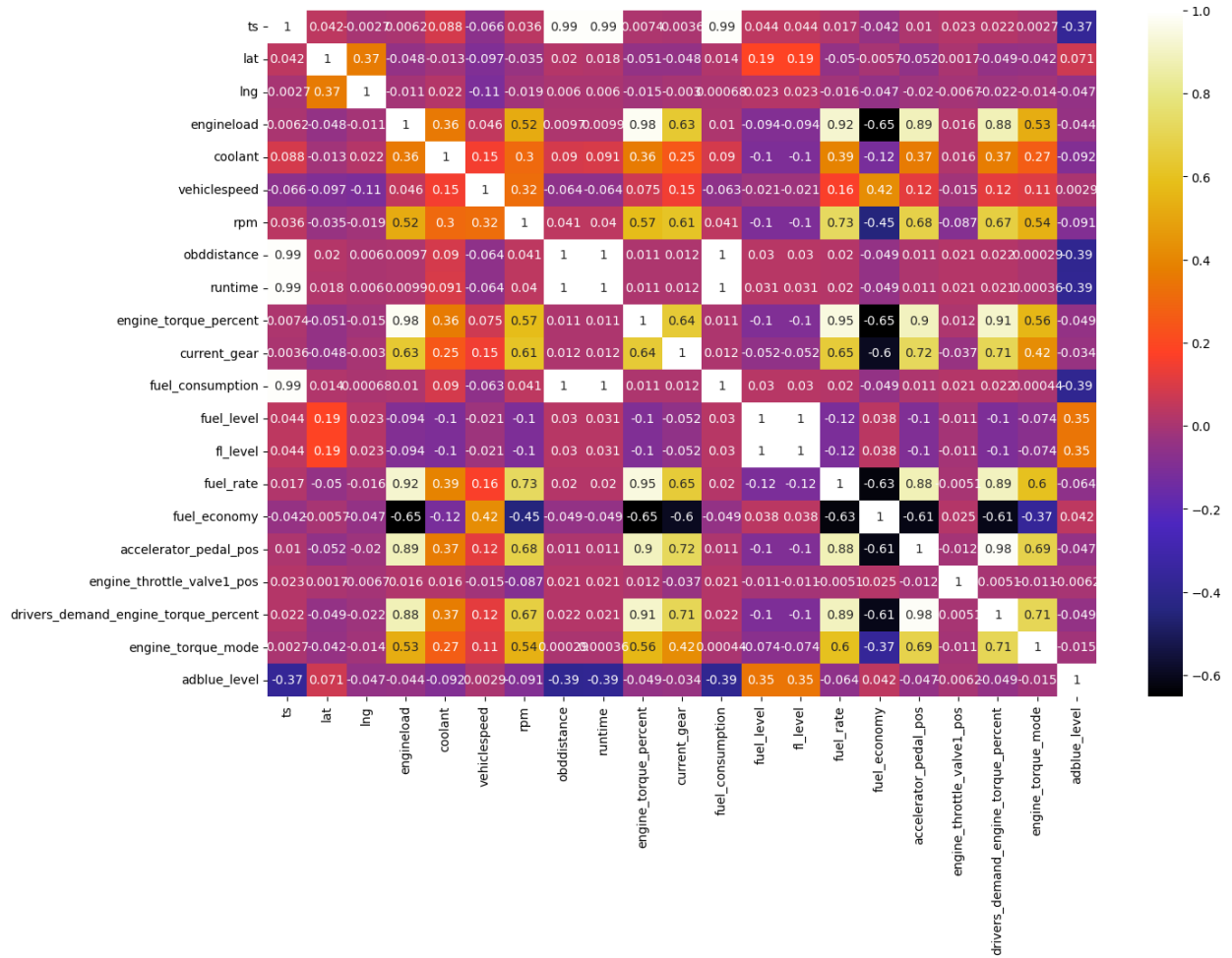
from scipy.stats import zscore
z_scores = df_5.apply(zscore)
df_no_o = df_5[(z_scores.abs() < 3).all(axis=1)] #removing outliers
using z-score

plt.figure(figsize=(10, 6)) #plotting graph for the cluster
plt.plot(df_no_o['vehiclespeed'], df_no_o['fuel_economy'], marker='*',
linestyle='-', color='b')
plt.title('Vehicle speed vs Fuel economy')
plt.xlabel('vehiclespeed')
plt.ylabel('fuel_economy')
```

```
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
df_corr=df_no_o.corr() #correlation matrix
plt.figure(figsize=(15,10))
sns.heatmap(df_corr, cmap="CMRmap", annot=True) #plotting heatmap
plt.show()
mask = np.tril(np.ones_like(np.abs(df_corr), dtype=bool), k=-1)
#masking lower triangle of the matrix
df_pairs = df_corr.where(mask & (np.abs(df_corr) > 0.4)).stack()
#forming pairs having absolute value of Correlation coefficient
greater than 0.4
pairs_with_corr = df_pairs.reset_index()
pairs_with_corr.columns = ['Feature1', 'Feature2', 'Correlation']
pairs_with_corr
```



```

0      Feature1 \
1      rpm
2      obddistance
3      runtime
4      runtime
5      engine_torque_percent
6      engine_torque_percent
7      current_gear
8      current_gear
9      fuel_consumption
10     fuel_consumption
11     fuel_consumption
12     fl_level
13     fuel_rate
14     fuel_rate
15     fuel_rate
16     fuel_rate
17     fuel_economy
18     fuel_economy

```

19		fuel_economy
20		fuel_economy
21		fuel_economy
22		fuel_economy
23		accelerator_pedal_pos
24		accelerator_pedal_pos
25		accelerator_pedal_pos
26		accelerator_pedal_pos
27		accelerator_pedal_pos
28		accelerator_pedal_pos
29	drivers_demand_engine_torque_percent	
30	drivers_demand_engine_torque_percent	
31	drivers_demand_engine_torque_percent	
32	drivers_demand_engine_torque_percent	
33	drivers_demand_engine_torque_percent	
34	drivers_demand_engine_torque_percent	
35	drivers_demand_engine_torque_percent	
36		engine_torque_mode
37		engine_torque_mode
38		engine_torque_mode
39		engine_torque_mode
40		engine_torque_mode
41		engine_torque_mode
42		engine_torque_mode

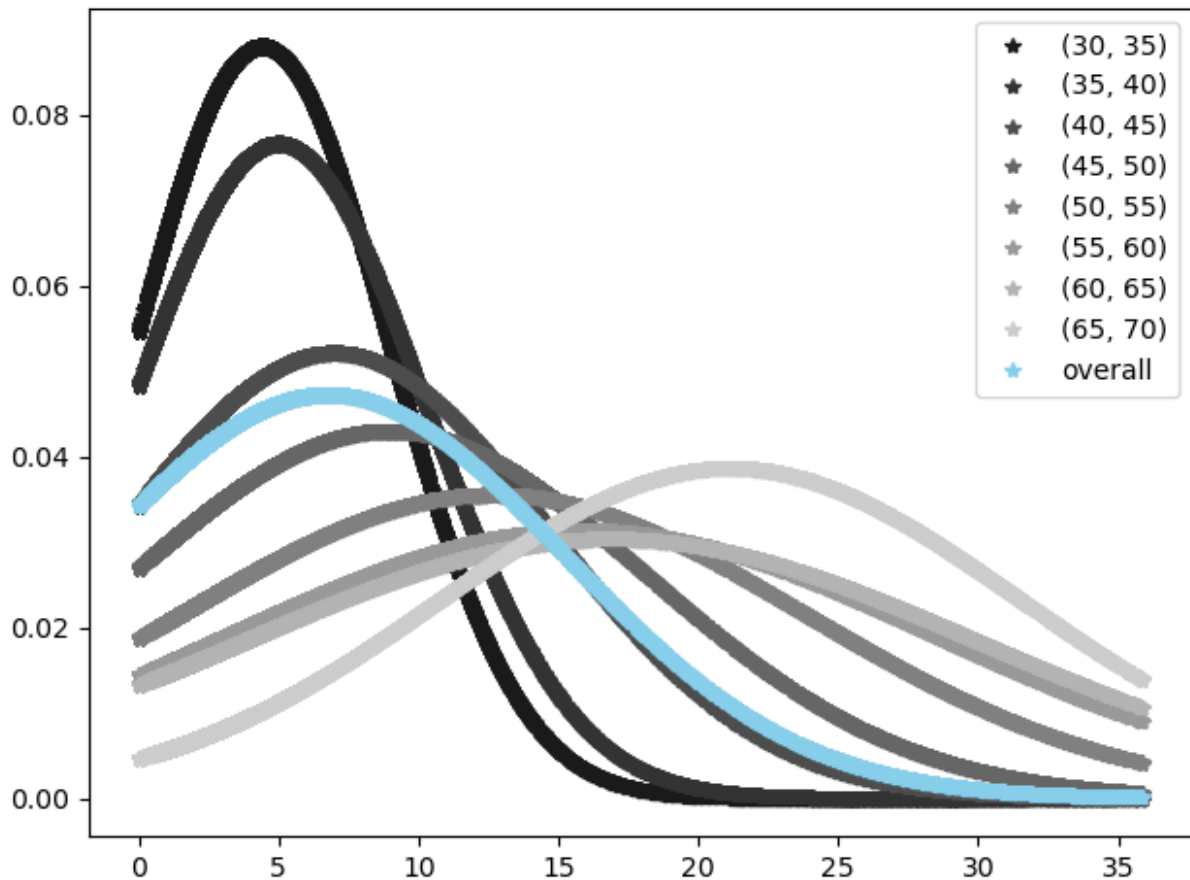
	Feature2	Correlation
0	engineload	0.521584
1	ts	0.993079
2	ts	0.991348
3	obddistance	0.998434
4	engineload	0.976730
5	rpm	0.574617
6	engineload	0.633190
7	rpm	0.607759
8	engine_torque_percent	0.643763
9	ts	0.992851
10	obddistance	0.999961
11	runtime	0.998439
12	fuel_level	1.000000
13	engineload	0.922113
14	rpm	0.726792
15	engine_torque_percent	0.948277
16	current_gear	0.648108
17	engineload	-0.648734
18	vehiclespeed	0.424910
19	rpm	-0.449403
20	engine_torque_percent	-0.651358
21	current_gear	-0.598719
22	fuel_rate	-0.627452

23	engineload	0.886277
24	rpm	0.682276
25	engine_torque_percent	0.895174
26	current_gear	0.722534
27	fuel_rate	0.883760
28	fuel_economy	-0.614356
29	engineload	0.880162
30	rpm	0.666568
31	engine_torque_percent	0.910829
32	current_gear	0.705606
33	fuel_rate	0.893075
34	fuel_economy	-0.609968
35	accelerator_pedal_pos	0.983489
36	engineload	0.527641
37	rpm	0.538904
38	engine_torque_percent	0.560599
39	current_gear	0.421902
40	fuel_rate	0.596160
41	accelerator_pedal_pos	0.685818
42	drivers_demand_engine_torque_percent	0.706362

```

from scipy.stats import norm #plotting normal distribution at
different intervals
means=[]
x=df_no_o['fuel_economy']
color=['0.1','0.2','0.3','0.4','0.5','0.6','0.7','0.8']
label=[]
for i in range(0,8):
    label.append((30+5*i,35+5*i))
for i in range(0,8):
    y = df_no_o['fuel_economy'].where((df_no_o['vehiclespeed']>(30+
(i*5))&(df_no_o['vehiclespeed']<(35+(i*5)))).dropna()
    mean, std_dev = norm.fit(y)
    means.append(mean)
    p = (1 / (np.sqrt(2 * np.pi) * std_dev)) * np.exp(-((x - mean)**2)
/ (2 * std_dev**2))
    plt.plot( x,p, label=label[i], marker='*', linestyle=' ',
color=color[i])
mean1, std_dev1 = norm.fit(x)
p1 = (1 / (np.sqrt(2 * np.pi) * std_dev1)) * np.exp(-((x - mean1)**2)
/ (2 * std_dev1**2))
plt.plot( x,p1, label= 'overall', marker='*', linestyle=' ',
color='skyblue')
plt.tight_layout()
plt.legend()
plt.show()

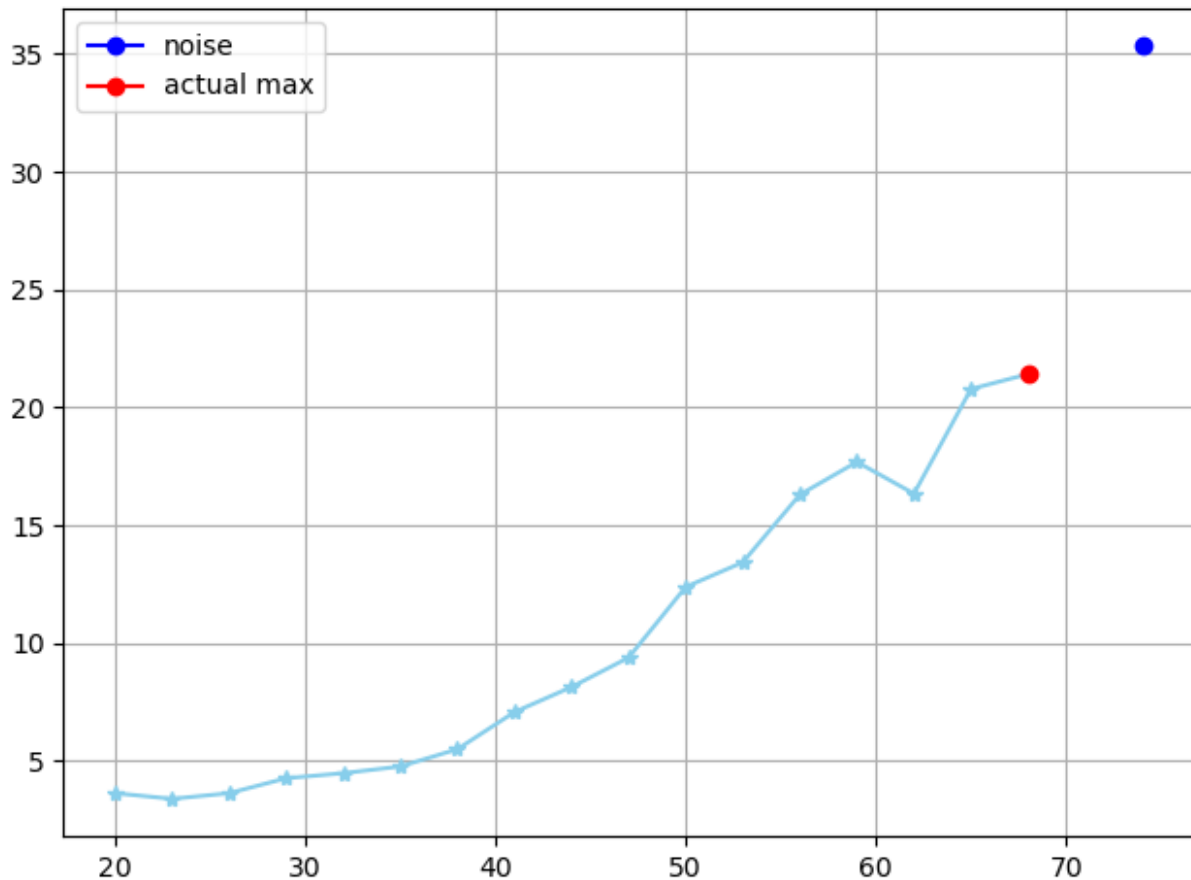
```



```

label1=[]
for i in range(0,20):
    label1.append(20+3*i)
means1=[]
for i in range(0,20):
    y =
df_no_o['fuel_economy'].where((df_no_o['vehiclespeed']>=(20+3*i))&(df_
no_o['vehiclespeed']<(23+3*i))).dropna()
    mean, std_dev = norm.fit(y)
    means1.append(mean)
plt.plot(label1,means1,marker='*',color='skyblue')
a=max(means1)
t=label1[means1.index(a)]
plt.plot(t,a, marker='o',label='noise', color='b')
plt.plot(label1[16],means1[16],label='actual max', marker='o',
color='r')
plt.tight_layout()
plt.legend()
plt.grid(True)
plt.show()

```



Observation-4:

1. Now we find a relation between vehicle speed and fuel economy.
2. As we increase speed, the fuel economy also increases until speed of 70km/h.

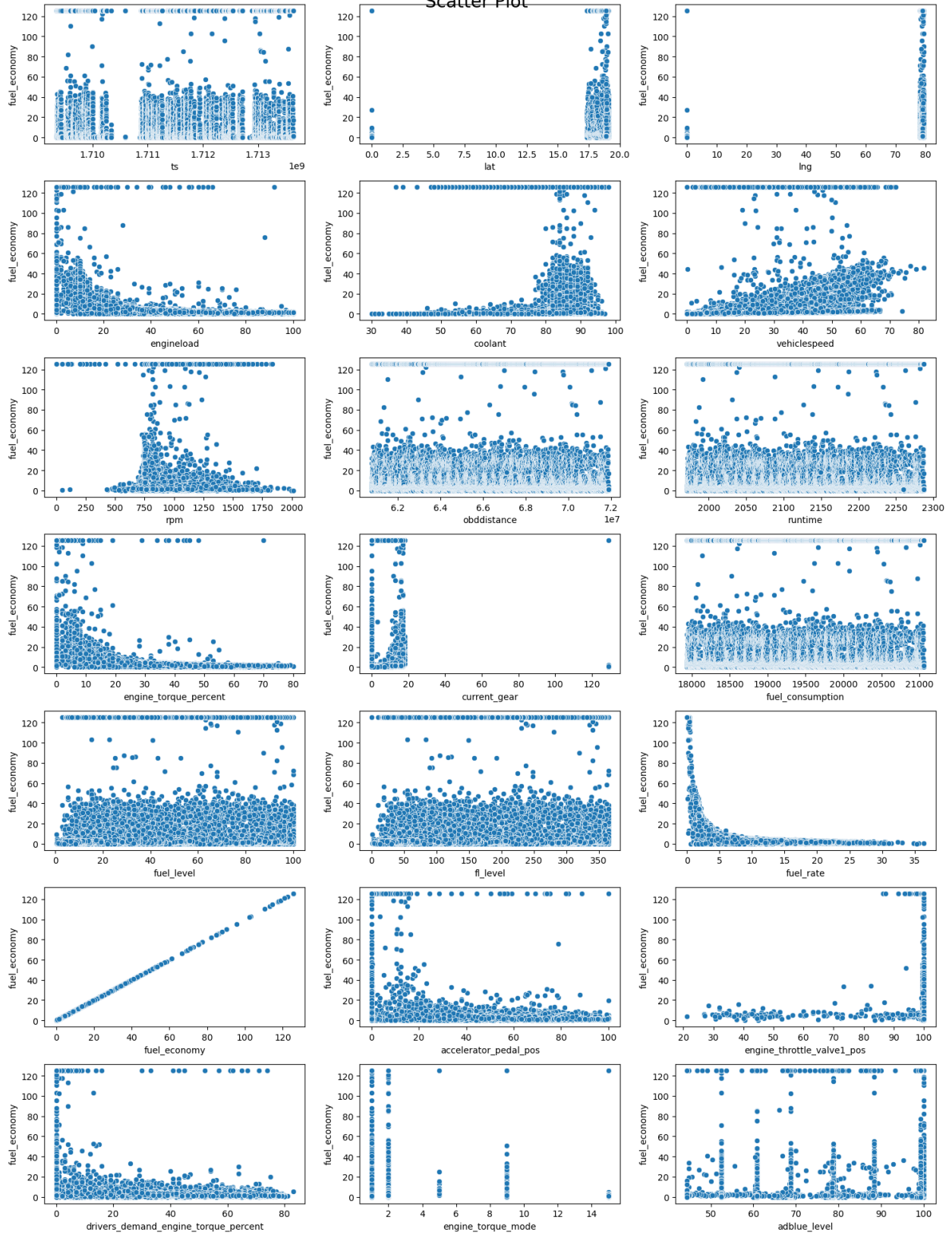
Inference:

1. At a speed of 65-70km/h the performance of vehicle is at the highest

```
#plotting scatter plot to see the relationship between numerical
features and target
#targeted feature is "fuel_economy"
#this is to check vehicle performance
#using whole cleaned dataframe
```

```
plt.figure(figsize=(15,20))
plt.suptitle('Scatter Plot', fontsize=20)
for i in range(0, len(num)):
    plt.subplot(7,3,i+1)
    sns.scatterplot(y='fuel_economy',x=num[i], data=df_2)
plt.tight_layout()
```

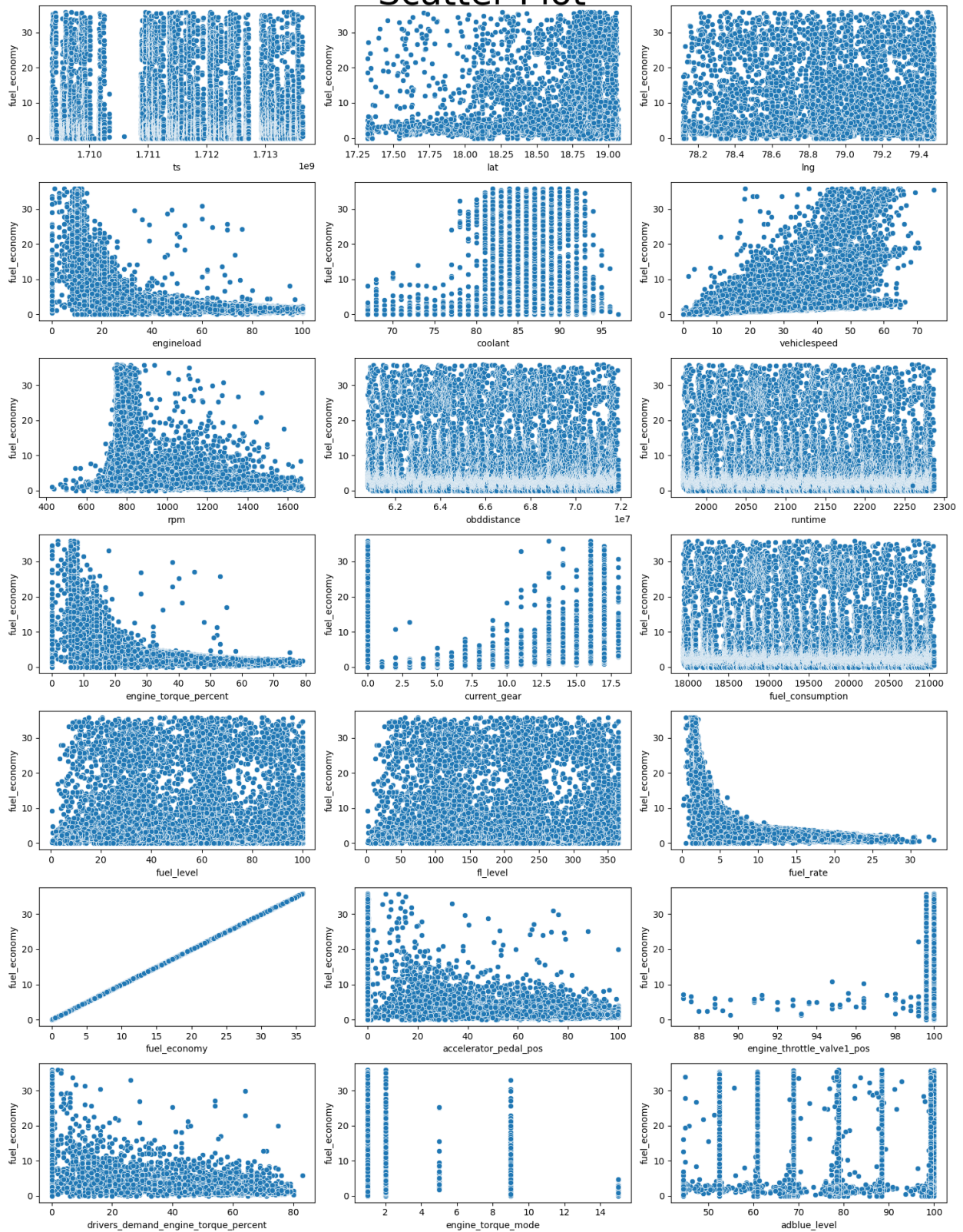
Scatter Plot



```
#using dataframe at cluster=1
```

```
plt.figure(figsize=(15,20))  
plt.suptitle('Scatter Plot', fontsize=40)  
for i in range(0, len(num)):  
    plt.subplot(7,3,i+1)  
    sns.scatterplot(y='fuel_economy',x=num[i], data=df_no_o)  
plt.tight_layout()
```

Scatter Plot



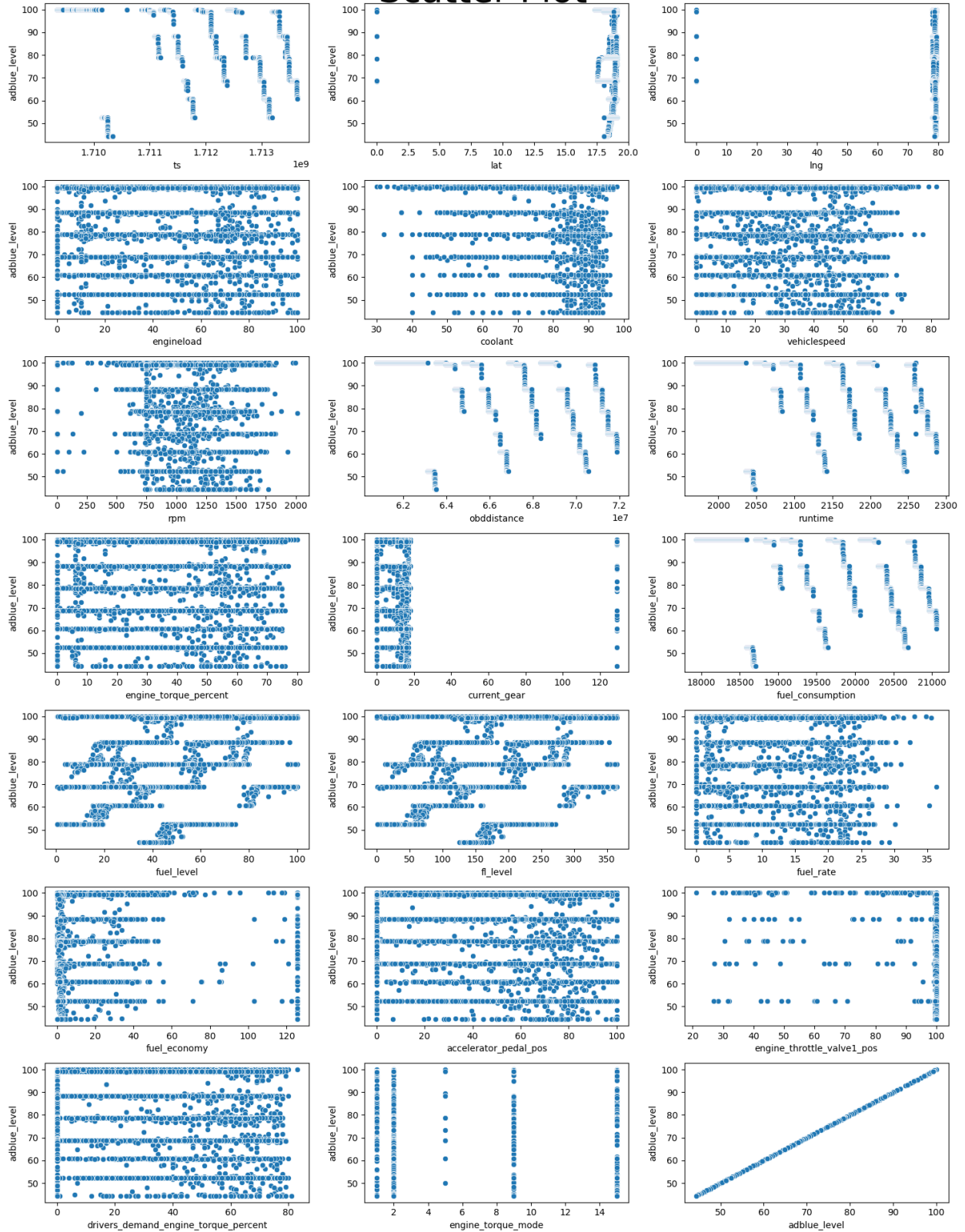
Observations-5:

1. fuel economy decreases as engine load increases, highest around 10-15
2. fuel economy increases as coolant increases, highest around 82-92
3. fuel economy increases as vehiclespeed increases, highest around 65-70
4. fuel economy is first increasing, highest around 800, then decreasing as rpm increases
5. fuel economy decreases as engine torque percent increases, highest around 5-10
6. fuel economy increases as current gear increases, highest around 16-17
7. fuel rate decreases as fuel economy increases
8. fuel economy decreases as accelerator pedal position increases
9. fuel economy is highest when engine throttle valve2 position is near 100
10. fuel econmoy decreases as drivers demand for engine torque percent increases, highest at 0
11. fuel economy has higher values at 3 modes of engine torque, at 1,2 and 9

```
#plotting scatter plot to see the relationship between numerical  
features and target  
#targeted feature is "adblue_level"  
#this is to check safety or emissions  
#using whole cleaned dataframe
```

```
plt.figure(figsize=(15,20))  
plt.suptitle('Scatter Plot', fontsize=40)  
for i in range(0, len(num)):  
    plt.subplot(7,3,i+1)  
    sns.scatterplot(y='adblue_level',x=num[i], data=df_2)  
plt.tight_layout()
```

Scatter Plot



Observations-6:

1. adblue level is considerably at higher levels when coolant is below 40
2. adblue level is considerably at higher levels when obd distance is below 6.3
3. adblue level is considerably at higher levels when fuel level is higher than 80
4. adblue level is considerably at higher levels when fuel economy is around 60-120

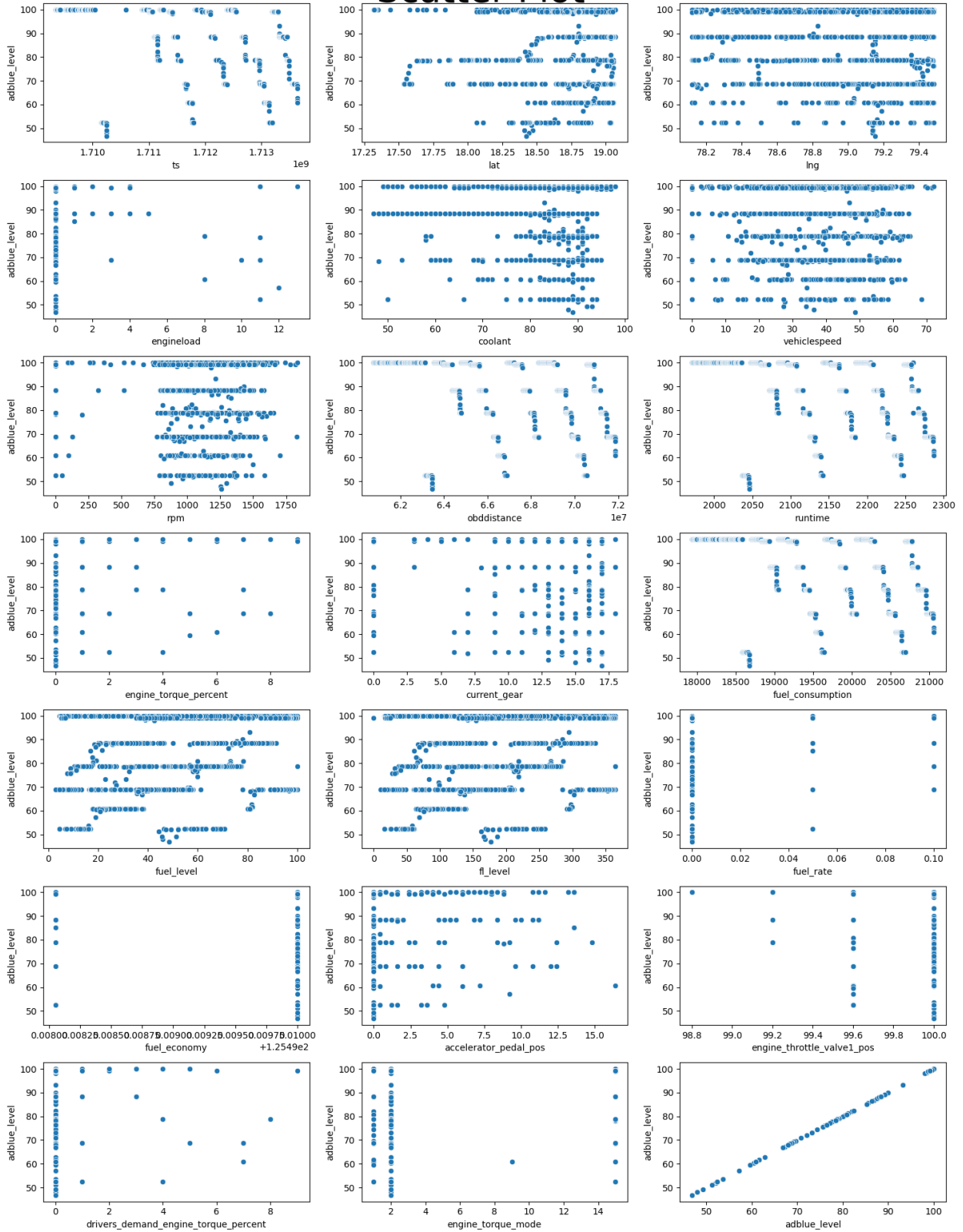
```
df_6=df_2.where(df_2['Cluster']==0).dropna()
del df_6['Cluster']

from scipy.stats import zscore
z_scores = df_6.apply(zscore)
df_no_o1 = df_6[(z_scores.abs() < 3).all(axis=1)]

#using dataframe at cluster=0

plt.figure(figsize=(15,20))
plt.suptitle('Scatter Plot', fontsize=40)
for i in range(0, len(num)):
    plt.subplot(7,3,i+1)
    sns.scatterplot(y='adblue_level',x=num[i], data=df_no_o1)
plt.tight_layout()
```

Scatter Plot



Observations-7:

1. adblue level is considerably at higher levels when rpm is between 50-750

COLLECTED OBSERVATIONS:

Observations-1:

1. runtime, fuel_consumption and obddistance has similar graphs and can be combined to form a new single feature mileage which is similar to fuel_economy.
2. drivers_demand_engine_torque_percent and accelerator_pedal_pos has similar graphs and can be combined to form a new feature driver_input_demand
3. fuel_level and fl_level are same thing and one can be omitted
4. engine_torque_percent and engineload has similar graphs and can be combined to form a new feature performance_demand

Observations-2:

1. brake_switch_status is mostly released
2. clutch_switch_status is mostly pressed
3. parking_switch_status is mostly released

Observations-3:

The graph shows there are 2 groups of data, so we need to separate them into 2 groups and then analyse

Observations-4:

1. Now we find a relation between vehicle speed and fuel economy.
2. As we increase speed, the fuel economy also increases until speed of 70km/h.

Inference:

1. At a speed of 65-70km/h the performance of vehicle is at the highest

Observations-5:

fuel economy decreases as engine load increases, highest around 2. 2. -15 fuel economy increases as coolant increases, highest around 83. 3. 2-92 fuel economy increases as vehicle speed increases, highest around 4. 4. 65-70 fuel economy is first increasing, highest around 800, then decreasing as engine torque percent increases, highest around 5. 5. reases fuel economy decreases as engine torque percent increases, highest around 6. 6. nd 5-10 fuel economy increases as current gear increases, highest around 7. 7. nd 16-17 fuel rate decreases as fuel economy 8. 8. increases fuel economy decreases as accelerator pedal position 9. 9. increases fuel economy is highest when engine throttle valve position 10. 10. is near 100 fuel economy decreases as drivers demand for engine torque percent highest at 11. highest at 0 fuel economy has higher values at 3 modes of engine torque and 9-6

Observations-6:

adblue level is considerably at higher level when coolant temperature is below 40 adblue level is considerably at higher levels when distance is below 6.3 adblue level is considerably at higher levels when fuel level is higher than 80 adblue level is considerably at higher levels when fuel economy is around 60-120-7

O-71. 1. bservations:

adblue level is considerably at higher levels when rpm is between 50-750

CONCLUSIONS AND RECOMMENDATIONS:

Conclusion-1: Feature Engineering and Redundancy

1. Combining Similar Features: --runtime, fuel_consumption, and obddistance have similar graphs and can be combined into a new feature mileage, representing vehicle efficiency. --drivers_demand_engine_torque_percent and accelerator_pedal_pos can be merged into driver_input_demand to simplify the representation of driver control. -- engine_torque_percent and engine_load can be merged into performance_demand to represent overall engine workload.
2. Removing Redundant Features: --As fuel_level and fl_level are almost identical, one of them should be omitted to avoid redundancy.

Conclusion-2: Switch Status Insights --Given their static nature, these switch statuses might not provide significant variation for predictive models. Consider their relevance in analysis or discard if unimpactful.

Conclusion-3: Clustering for Segmentation --The data exhibits two distinct groups, indicating potential clusters. These groups could correspond to different vehicle operating conditions or driver behaviors.

Conclusion-4: Speed and Fuel Economy Relationship --Optimize driving strategies and algorithms to maintain speeds within the range of 65-70 km/h for enhanced fuel efficiency and performance.

Conclusion-5: Factors Influencing Fuel Economy that in turn tells about vehicle performance -- Optimize vehicle operation around the identified high fuel economy ranges and minimize conditions leading to decreased efficiency.

Conclusion-6: AdBlue Level Analysis --Monitor and control AdBlue levels under these conditions to maintain emission standards and system efficiency.

Conclusion-7: RPM and AdBlue Levels --Investigate why AdBlue consumption is higher at low RPM and explore strategies to optimize its use and reduce harmful emissions